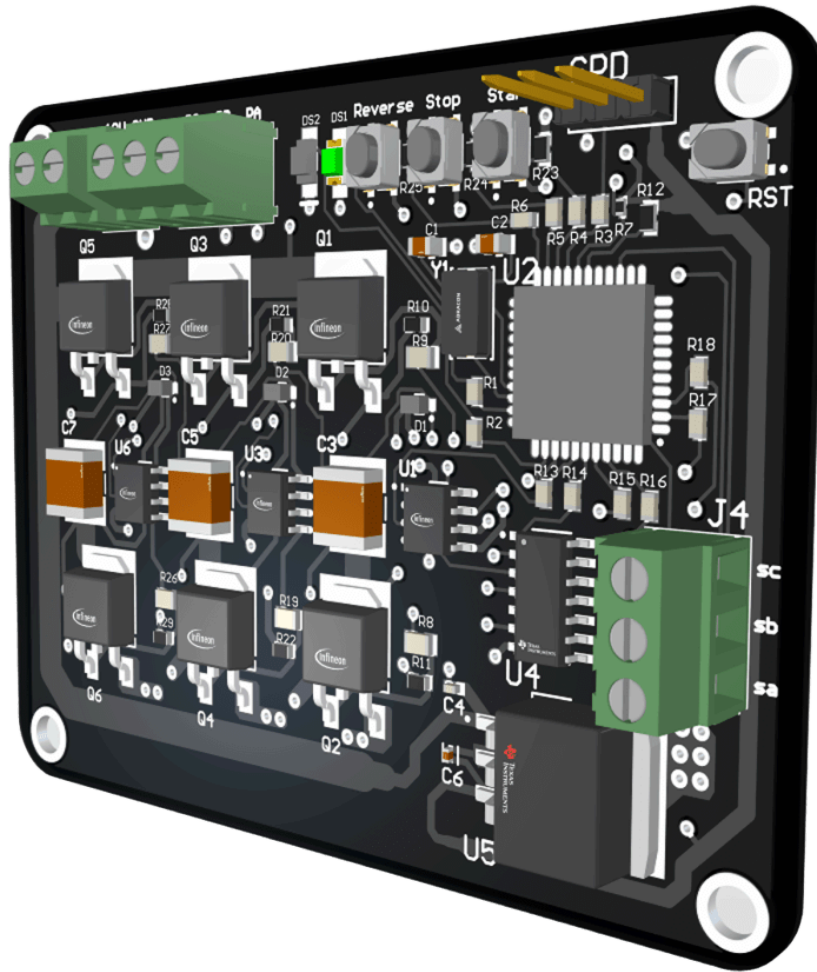




Design of a 3-Phase BLDC Motor Controller

28.06.2025

Fawaz Yousif

Overview

This project idea was simulating and designing a **3-phase Brushless DC (BLDC) motor controller**, also known as an **ESC(Electronic Speed Controller)**, using a **PIC18F microcontroller** with **IR2101 gate drivers** and **Hall-sensor-based 6-step commutation**. The objective is to control the speed and direction of a BLDC motor using **PWM** for speed modulation and **logic switching** for phase control, all simulated and verified in **Proteus** before hardware implementation.

The system accepts **potentiometer input** to vary motor speed and uses **START, STOP, and REVERSE buttons** for user control, aiming for a customizable, low-cost controller suitable for robotics and small vehicle applications.

Theoretical Explanation

How does a BLDC Motor work?

A **Brushless DC (BLDC) motor** is a **synchronous electric motor that uses electronic commutation** instead of mechanical brushes and a commutator. Its structure mainly consists of both the **Rotor**, which is the rotating permanent magnet part, and the **Stator**, which is the stationary part that is formed by the 3-phase windings.

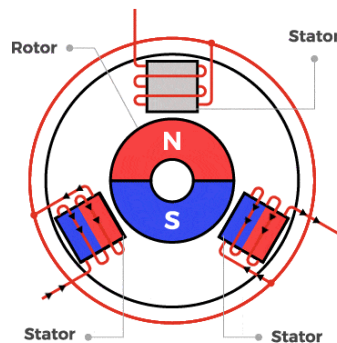


Figure 1: BLDC Motor Structure

Source: espressif.com

Principle of Operation:

Initially the stator windings are energized in a specific sequence to produce a **rotating magnetic field**. Then the permanent magnet rotor follows the rotating field,

creating **mechanical rotation**. Unlike AC induction motors, **BLDC motors use rotor position feedback** (Hall sensors) to control commutation precisely. The sequence of energizing the stator is controlled by what is called a 6-step commutation.

What is a 6-step Commutation?

In a 6-step commutation, one **electrical cycle** is divided into **six steps**, each **60°** apart. At each step two out of three phases are energized (one high, one low) and the third phase is left floating. This eventually produces a **rotating magnetic field**, pulling the rotor forward.

Step	Hall Sensors			Phases Energized
	A	B	C	
1	1	-1	0	A+, B-
2	1	0	-1	A+, C-
3	0	1	-1	B+, C-
4	-1	1	0	A-, B+
5	-1	0	1	C+, A-
6	0	-1	1	B-, C+

Table 1: Sequence of the 6-Step Commutation

The **sequence of energizing phases** is determined by **reading the Hall sensor signals**, which indicate the rotor position in real-time. What if we would like to control the speed of the motor? Here comes what is known as a PWM.

What is a PWM?

PWM or **Pulse Width Modulation** is a signal generated by the microcontroller to **control the speed** of the motor. By adjusting the "on" time (**duty cycle**) of the pulses, the

controller effectively changes the perceived voltage, thereby controlling the motor's speed. A **higher duty cycle** increases the **average voltage applied to the motor**, resulting in **higher speed**.

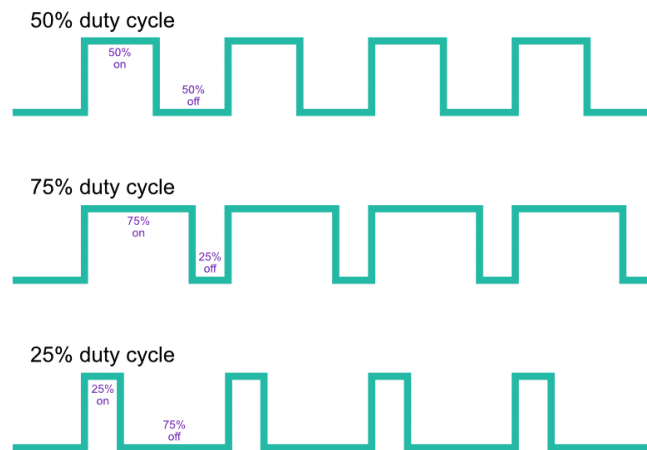


Figure 2: Example of PWM Signals with Their Duty Cycles

Simulation

Proteus was used to simulate the full BLDC control system, as it is capable of supporting co-simulation of microcontroller firmware with hardware circuits. The simulation mainly included the PIC18F46K22, IR2101 gate drivers, MOSFET bridge, Hall sensors, and the BLDC motor model.

Simulation Objectives

The goal of the simulation was to verify PWM generation and duty cycle variation using a virtual potentiometer. As well as Validating the six-step commutation sequence and correct phase energizing. And Confirming the response of the system to Hall sensor changes.

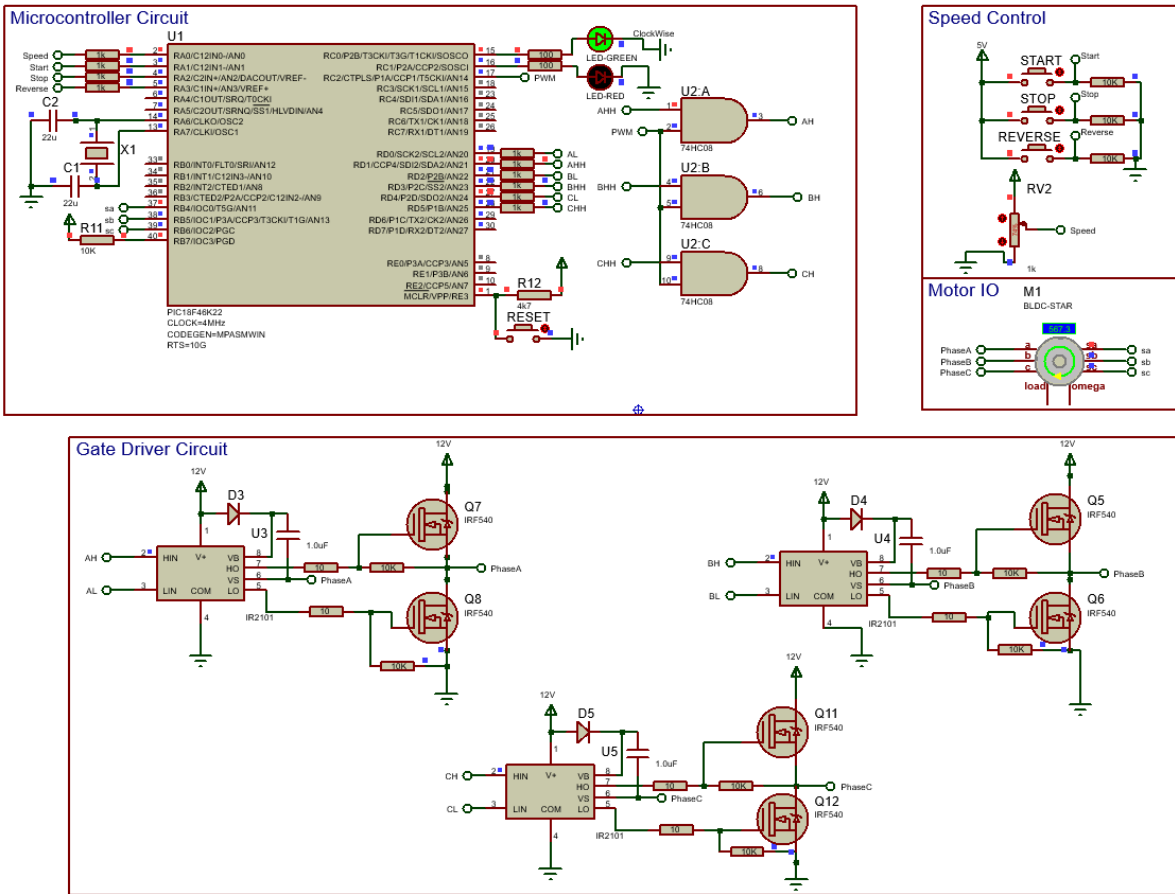


Figure 3: Simulation Schematic

PIC Microcontroller Code Logic:

The code sets up the PIC18F46K22 to use PWM for speed control, ADC (Analog to Digital Converter) to read a potentiometer, and buttons to start, stop, or reverse the motor. It also prepares an interrupt that checks the Hall sensors to know the motor's rotor position and chooses the right phases to turn on for forward or reverse rotation using lookup tables.

In the main loop, when the start button is pressed, the motor runs and LEDs show the direction. The potentiometer changes the PWM duty cycle, adjusting the motor speed. The interrupt keeps updating which motor phases to energize based on the rotor position so the motor keeps turning while the PWM controls how fast it spins.

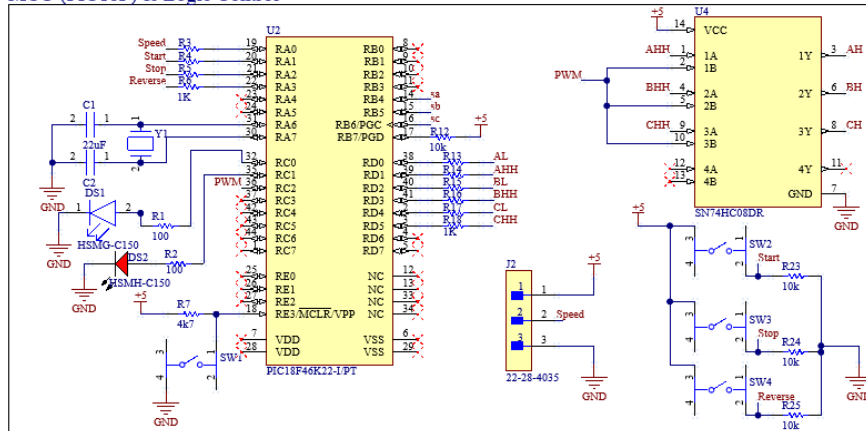
Electronic Design

I. The Schematic Diagram

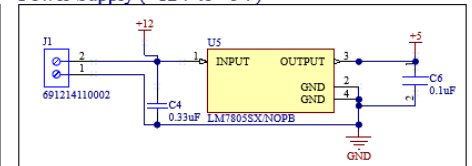
The electronic design has included the following:

- **PIC18F46K22 microcontroller** for core control logic.
- **74HC08 Logic AND gates**: combine PWM with commutation signals for each phase.
- **IR2101 gate drivers** to drive the high-side and low-side MOSFETs.
- **MOSFET bridge** to control the 3-phase outputs to the BLDC motor.
- **Hall-effect sensor inputs** for rotor position feedback.
- **Potentiometer** connected to the ADC for speed control.
- **Start, Stop, Reverse and Reset buttons** for user interaction.
- **Bootstrap capacitors, decoupling capacitors, gate resistors, and protection diodes** for stability and reliability.

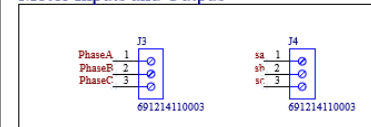
MCU (PIC18F) & Logic Control



Power Supply (+12V to +5V)



Motor Inputs and Output



Gate Driver Circuit

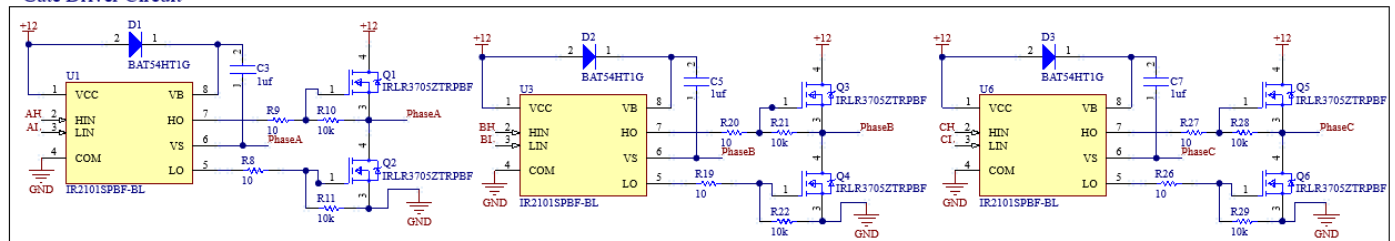


Figure 4: The Schematic Diagram

II. PCB Layout

A **4-layer PCB (2 signal layers + 2 ground planes)** was designed in Altium, with the consideration of a clear separation between high-current MOSFET sections and low-voltage control logic. The 2 **dedicated ground planes** on inner layers minimize noise, reduce EMI, and improve signal integrity for PWM and Hall signals.

Another consideration has been taken into account is the **wide power traces** for +12V and ground to handle motor currents safely. Also **decoupling capacitors** have been considered to be positioned near the MCU and gate drivers to maintain stable operation.

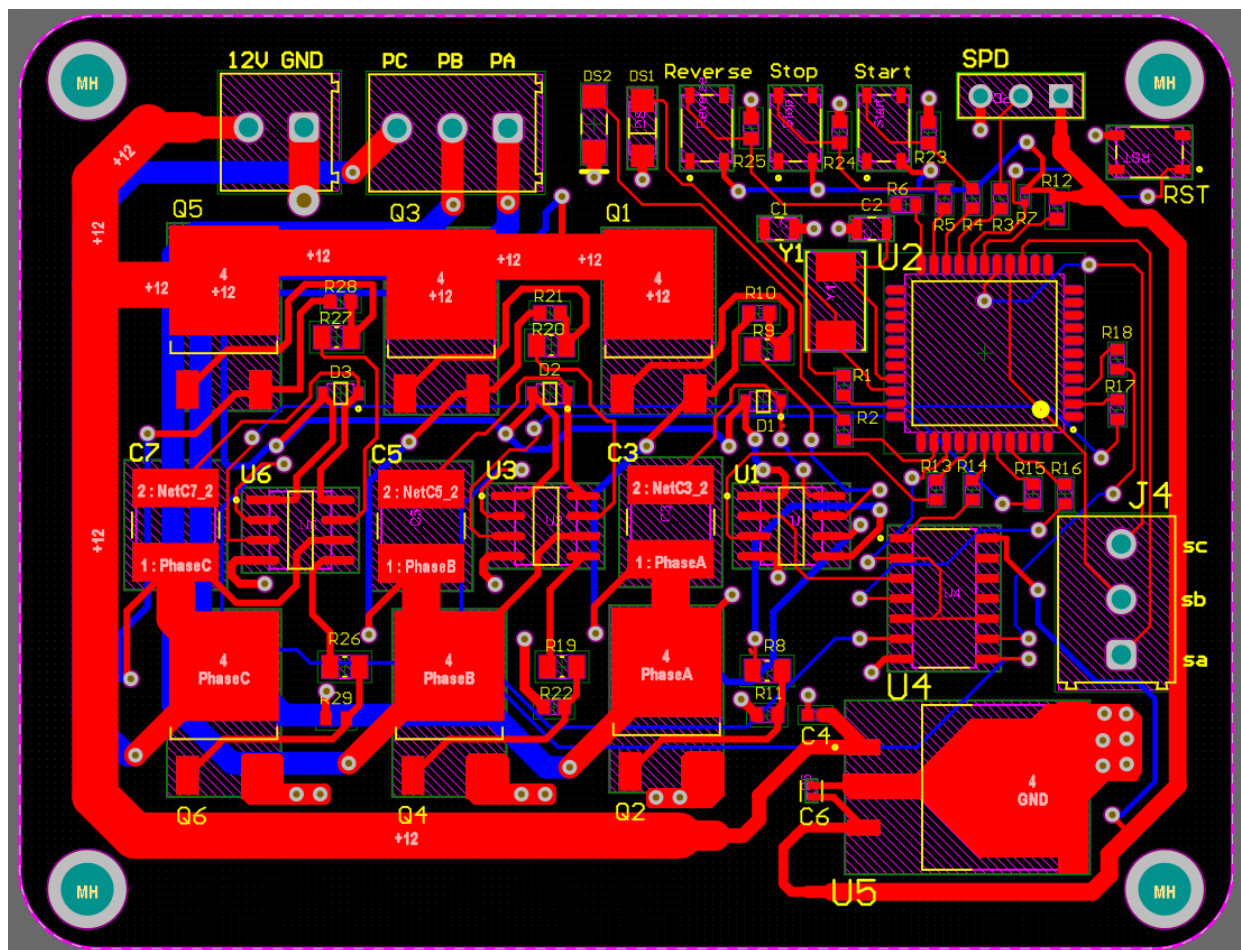


Figure 5: 2d View of the PCB Layout

Project Repository and Profile Links

This project is open source hardware, and you can access its firmware and hardware files through the following links:

-  **GitHub Repository:** <https://github.com/Fawwazoa/bldc-motor-controller>
(Contains firmware code, schematic, PCB layout, BOM, and project documentation)
-  **My Hackaday Profile:** hackaday.io/Fawwazoa
(For project logs, community discussions, and build updates.)